

Step 3 — The Writer Agent Synthesizes a Final Report — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (the real chain and prompt), **Advanced** (what makes a synthesis prompt actually synthesize, instead of just concatenate). Read Basic first even if you already built Project 4's Writer — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — Plain chain, no tools, no MCP](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — Plain chain, no tools, no MCP

Basic Version

The Writer is the simplest piece in this whole project. It doesn't connect to anything — no MCP server, no `stdio_client`, nothing `async`. It's exactly Project 4 Step 1's Writer chain: a prompt goes in, text comes out.

Things to use: - A prompt template with two placeholders: one for the notes findings, one for the web findings. - `ChatOpenAI` (or the plain `openai` client) to actually generate the report text. - A plain Python function, no `async def` anywhere in this file.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

`run_writer(notes_findings, web_findings) -> str` builds one prompt string (or message list) containing both findings, sends it to the model once, and returns the response text. No loop, no tool schema, no `max_iterations` — none of that applies here, because there's nothing for the model to call.

The exact pieces: - A prompt that clearly labels each source: `f"Notes findings:\n{notes_findings}\n\nWeb findings:\n{web_findings}"` — labeling them explicitly matters, so the model (and a human reading the prompt later) can tell which fact came from which source. - An instruction in the system or user prompt that says explicitly: combine these into one short report; don't just repeat each source in its own paragraph. - `notes_agent_node / web_agent_node` 's existing shape from Step 2 is the template for `writer_agent_node` too — read from state, call the plain function, write the result into state, return a `Command` .

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Think about what "don't just concatenate" actually needs in the prompt to reliably happen. A vague instruction like "summarize these" tends to produce exactly the concatenated-paragraphs result you're trying to avoid — the model plays it safe and restates each source separately, because that's the lowest-risk way to "not lose information." A more specific instruction works better: ask for a short report *organized by topic or conclusion*, not by source — e.g. "State what is known about Project Atlas's status, citing whichever source(s) support each point, rather than writing one paragraph per source." That framing forces the model to actually merge the information instead of just relaying it twice.

Also think about the Supervisor's routing change this step needs. Up through Step 2, the Supervisor routed to `END` once every needed specialist had run. Now it needs one more state: once every needed specialist has run, route to `"writer_agent"` instead — and `writer_agent_node` is the one that finally routes to `END`. Get this ordering right, or the Writer either never gets called, or gets called before a specialist it needed has actually finished.

One more real case worth testing now: what does the Writer produce if only one specialist ran (task 1 or task 2, which only need one source)? The prompt should handle an empty `web_findings` (or `notes_findings`) gracefully — a report that only had one real source shouldn't awkwardly reference a "web findings" section that says nothing, or falsely imply the web wasn't checked when it simply wasn't needed for that task.

Things to try before Hint 2: - Run the Writer directly (not through the graph yet) on made-up notes findings and made-up web findings that slightly *disagree* with each other — see whether your prompt handles that honestly (naming the discrepancy) or quietly picks one and ignores the other.

Difference between Basic, Intermediate, and Advanced: Basic names the shape (prompt in, text out, no tools). Intermediate gives the real function signature and prompt structure. Advanced is about the actual synthesis instruction working — a specific "organize by conclusion, not by source" framing, correct Supervisor sequencing so the Writer runs last, and handling the single-source case gracefully — which is the difference between a Writer that technically runs and one that actually produces the coherent report this step promises.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
run_writer(notes_findings, web_findings):
    build a prompt with both findings labeled clearly
    tell the model: combine into one short report, organized by topic, not by source
    send it to the model once
    return the text

writer_agent_node(state):
    call run_writer(state.notes_findings, state.web_findings)
    save result into state["report"]
    route to END

supervisor_node(state):
    same as Step 2, but once every needed specialist has run:
        route to writer_agent instead of END
```

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
# writer_agent.py
from openai import OpenAI

client = OpenAI()

WRITER_PROMPT = """You are writing a short status report from research findings.

Notes findings:
{notes_findings}

Web findings:
{web_findings}

Write one short, clear report. State what is known, citing which source(s)
support each point. Do not write one paragraph per source -- organize by
topic or conclusion instead."""

def run_writer(notes_findings: str, web_findings: str) -> str:
    prompt = WRITER_PROMPT.format(
        notes_findings=notes_findings or "(not checked for this request)",
        web_findings=web_findings or "(not checked for this request)",
    )
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": prompt}],
    )
    return response.choices[0].message.content
```

Your turn: write `writer_agent_node`, and update `supervisor_node` to route to `"writer_agent"` once all needed specialists have run, then compare against the [Solution](#).

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Fill in the Supervisor's updated routing yourself:

```
# writer_agent.py (node function, added)
from langgraph.graph import END
from langgraph.types import Command
from state import TeamState
# run_writer(notes_findings, web_findings) -> str already exists above

def writer_agent_node(state: TeamState) -> Command:
    report = run_writer(state.get("notes_findings", ""), state.get("web_findings"))
    return Command(goto=END, update={"report": report})
```

```
# supervisor.py (updated)
from langgraph.graph import END
from langgraph.types import Command
from state import TeamState

def figure_out_needed(task: str) -> list[str]:
    # unchanged from Step 2
    ...

def supervisor_node(state: TeamState) -> Command:
    needed = state.get("needed") or figure_out_needed(state["task"])
    visited = state.get("visited") or []

    for specialist in needed:
        if specialist not in visited:
            return Command(goto=specialist, update={"needed": needed})

    # your turn: every needed specialist has run -- route to "writer_agent"
    # instead of END, the same way Step 2 routed to a specialist
    ...
```

Compare your finished `supervisor.py` and `writer_agent.py` against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both assume the happy path — both specialists ran, both findings are real text. Advanced adds the `or "(not checked for this request)"` fallback for tasks that only needed one specialist, and shows the exact Supervisor change (routing to `"writer_agent"` instead of `END` once specialists are done) that makes the Writer actually the *last* thing that runs, not a separate step you call by hand.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)